While Laravel's Query Builder covers most use cases, sometimes you need to use raw SQL for complex queries or special SQL expressions. Laravel provides `DB::raw()` for such scenarios, allowing you to write raw expressions within your query.

11.4.1 Selecting with Raw Expressions

```
$users = DB::table('users')
           ->select(DB::raw('count(*) as total_users'))
           ->get();
```

You can also alias fields or perform calculations:

```
$results = DB::table('orders')
             ->select(DB::raw('SUM(total) as total_sales'))
             ->where('status', 'completed')
             ->get();
```

11.4.2 Using Raw in Where Clauses

```
$users = DB::table('users')
           ->whereRaw('YEAR(created_at) = ?', [2024])
           ->get();
```

11.4.3 Order By with Raw

```
$users = DB::table('users')
           ->orderByRaw('LENGTH(name) ASC')
           ->get();
```

While raw expressions offer flexibility, they should be used cautiously to avoid SQL injection. Always pass dynamic values as bindings rather than injecting them directly into raw strings.

11.5 Introduction to the Repository Pattern

As your Laravel application grows, separating business logic from data access becomes increasingly important. This is where the **Repository Pattern** comes in. The Repository Pattern provides a clean abstraction between your application's business logic and the underlying data source, whether it's a database, API, or something else.

Instead of writing queries directly in your controllers or services, you define a repository class that acts as a mediator between your models or query builders and